

Smart Hospital Priority & Routing System

Complete Function, String & Structure Reference

Course: CS222IA – Principles of Programming using C
RV College of Engineering, Bengaluru | PBL Mini Project – Part B

Table of Contents

1. The Patient Struct
2. Global Variables
3. String & Time Library Functions
 - strcpy
 - strcmp
 - strstr
 - strcspn
 - strlen
 - strftime
4. Standard I/O Functions
 - printf / fprintf
 - scanf / sscanf
 - fgets
 - fopen / fclose
 - getchar
5. Program-Defined Functions
 - getCurrentTimestamp
 - calculateRiskScore
 - classifyPriority
 - routeDepartment
 - logToFile
 - loadFromFile
 - rewriteLogFile
 - bubbleSortByScore
 - addPatient
 - displayAllPatients
 - viewLogFile

- `searchPatientByName`
- `deletePatientByID`
- `deleteAllRecords`
- `deleteMenu`
- `displayMenu`
- `main`

6. C Concepts Summary

1. The Patient Struct

A **struct** (structure) is a user-defined data type that groups related variables of different types under one name. Without it, you would need 11 separate arrays for 11 fields — making the code unmanageable.

```
struct Patient {
    int    id;           // unique patient number
    char   name[50];     // patient name (max 49 chars + '\0')
    int    age;
    float  temperature;  // body temp in Celsius
    int    bp;           // systolic blood pressure mmHg
    int    pulse;        // heart rate bpm
    char   symptom[100];
    int    riskScore;    // calculated penalty score
    char   priority[20]; // "EMERGENCY" / "URGENT" / "NORMAL"
    char   department[50]; // routed department name
    char   timestamp[30]; // "YYYY-MM-DD HH:MM:SS"
};
```

Field	Type	Why that type
id	int	Whole numbers only; no decimals needed.
name, symptom, priority, department, timestamp	char[]	C has no native <i>string</i> type; strings are null-terminated character arrays.
temperature	float	Decimal precision needed (e.g. 38.5 °C).
bp, pulse, age, riskScore	int	Whole-number measurements.

Pointer to struct (`struct Patient *p`): Several functions receive a pointer to a `Patient` so they can modify it directly. Use `p->field` (arrow operator) to access fields through a pointer, or `(*p).field` — both are equivalent.

2. Global Variables

```
struct Patient patients[MAX_PATIENTS]; // array of up to 200 Patient records
int numPatients = 0;                  // how many slots are currently used
int maxID      = 0;                  // highest ID ever assigned
```

Variable	Purpose
patients[]	Single array that holds both records loaded from disk AND patients added in the current session.
numPatients	Acts as both a count and the index of the next free slot (patients[numPatients] = p; numPatients++;).
maxID	Ensures new IDs are always unique even when old records were deleted (creating gaps like 1, 3, 4). New patients get maxID + 1 .

3. String & Time Library Functions

`string.h` **`strcpy(destination, source)`**

Copies every character from `source` to `destination`, including the null terminator `'\0'`.

Why not use `=` ? In C, arrays cannot be assigned with `=`. Writing `p.priority = "EMERGENCY"` is a compile error. `strcpy` is the correct way.

```
strcpy(p->priority, "EMERGENCY");    // classifyPriority()
strcpy(p->department, "Cardiology"); // routeDepartment()
strcpy(sym, p->symptom);              // copy before modifying (safe buffer)
strcpy(nameLower, patients[i].name); // copy before lowercasing
```

Parameter	Meaning
destination	char array that receives the string
source	string to copy from (can be a literal or another array)

`string.h` **`strcmp(string1, string2)`**

Compares two strings character by character and returns an integer.

Return value	Meaning
0	Both strings are identical
> 0	string1 comes after string2 alphabetically
< 0	string1 comes before string2 alphabetically

Why not use `==` ? `==` on char arrays compares *memory addresses*, not content. `strcmp` compares the actual characters.

```
if (strcmp(p.priority, "EMERGENCY") == 0) ... // addPatient() colour selection
if (strcmp(confirm, "YES") != 0) ...         // deleteAllRecords() safety
check
```

`string.h` **`strstr(haystack, needle)`**

Scans `haystack` for the first occurrence of `needle` .

Return value	Meaning
non-NULL pointer	needle found (truthy in an if-condition)
NULL	needle not found (falsy)

```
if (strstr(sym, "chest") || strstr(sym, "heart"))
    strcpy(p->department, "Cardiology");          // routeDepartment()

if (strstr(nameLower, query) != NULL) { ... }      // searchPatientByName()
```

This allows *partial, keyword-based* matching — a patient who types "severe chest tightness" still routes to Cardiology because `strstr` finds "chest" anywhere inside the string.

`string.h` **`strcspn(string, reject_set)`**

Returns the number of characters at the start of `string` that do *not* appear in `reject_set` . Effectively returns the index of the first matching character.

Why needed with `fgets`? `fgets` reads the `'\n'` (newline) into the buffer. If not removed, it becomes part of the name or confirmation string, causing subtle bugs.

```
fgets(p.name, 50, stdin);
p.name[strcspn(p.name, "\n")] = '\0';
// "Rajesh\n" → strcspn returns 6 → p.name[6] = '\0' → "Rajesh"
```

Used after **every** `fgets` call in the program: patient name, chief symptom, search query, delete confirmation.

`string.h` **`strlen(string)`**

Returns the number of characters in a string, *not* counting the null terminator.

```
strlen("hello") → 5
strlen("")      → 0
```

Used in `loadFromFile()` to trim trailing spaces that the fixed-width format specifier `%-20s` pads into fields:

```
int len = (int)strlen(p.name) - 1;
while (len >= 0 && p.name[len] == ' ') {
    p.name[len] = '\\0';
    len--;
}
```

time.h **strftime(buffer, size, format, tm_struct)**

Formats a `struct tm` (broken-down time) into a human-readable string according to a format string.

Format code	Meaning	Example
%Y	4-digit year	2026
%m	2-digit month	05
%d	2-digit day	22
%H	Hour (24h)	14
%M	Minutes	30
%S	Seconds	07

```
strftime(buffer, 30, "%Y-%m-%d %H:%M:%S", t);
// Result: "2026-05-22 14:30:07"
```

4. Standard I/O Functions

stdio.h **printf / fprintf**

`printf(format, ...)` prints formatted text to the **terminal**.

`fprintf(fp, format, ...)` does the same but writes to a **file**.

```
printf("Name: %s | Age: %d | Temp: %.1f\n", p.name, p.age, p.temperature);
fprintf(fp, "[%s] ID:%d | Name:%-20s ...\\n", p.timestamp, p.id, p.name);
```

Format specifier	Meaning
------------------	---------

%d	int
----	-----

%f / %.1f	float (%.1f = 1 decimal place)
-----------	--------------------------------

%s	string (char array)
----	---------------------

%-20s	left-aligned string in a 20-char wide field (padded with spaces)
-------	--

%3d	right-aligned int in a 3-char wide field
-----	--

stdio.h **scanf / sscanf**

`scanf(format, &var)` reads formatted input from the **keyboard**.

`sscanf(string, format, &var)` reads formatted input from a **string in memory** — used in `loadFromFile()` to parse each line of the log file.

```
scanf("%d", &targetID);    // read int from keyboard

int parsed = sscanf(line,
    "[%[^]] ID:%d | Name:%20[^|] ...",
    p.timestamp, &p.id, p.name, ...);
// %[^\]] reads characters until ']' is found
// %[^|] reads characters until '|' is found
```

Why sscanf? The log file uses a fixed format written by `fprintf`. `sscanf` can reverse-parse each line using the same template, reconstructing each `Patient` field from its text representation.

stdio.h **fgets(buffer, size, stream)**

Reads at most `size - 1` characters from `stream` into `buffer`, stopping at `'\n'` or EOF. Always null-terminates. The `'\n'` is kept in the buffer.


```
fgets(p.name, 50, stdin);          // read from keyboard
fgets(line, sizeof(line), fp);     // read one line from file
```

Why fgets and not scanf for strings? `scanf("%s", ...)` stops at whitespace, so names like "Ravi Kumar" would only read "Ravi". `fgets` reads the entire line including spaces.

stdio.h **fopen / fclose**

`fopen(filename, mode)` opens a file and returns a `FILE *` pointer. Always check for `NULL` (file could not be opened). `fclose(fp)` flushes and closes the file.

Mode	Behaviour	Used in
"a"	Append — adds to end, never deletes existing content	<code>logToFile()</code>
"r"	Read — file must exist	<code>loadFromFile()</code> , <code>viewLogFile()</code>
"w"	Write — creates file or truncates (empties) existing file	<code>rewriteLogFile()</code> , <code>deleteAllRecords()</code>

stdio.h **getchar()**

Reads one character from `stdin`. Used to consume the leftover `'\n'` that `scanf("%d", ...)` leaves in the input buffer, preventing `fgets` from immediately reading an empty line.

```
while (getchar() != '\n'); // drain buffer until newline is consumed
```

5. Program-Defined Functions

getCurrentTimestamp(char *buffer)

void Writes the current date and time as a formatted string into `buffer` .

```
void getCurrentTimestamp(char *buffer) {
    time_t now = time(NULL);           // seconds since Jan 1 1970
    struct tm *t = localtime(&now);    // convert to local broken-down time
    strftime(buffer, 30, "%Y-%m-%d %H:%M:%S", t);
}
```

Call

Purpose

<code>time(NULL)</code>	Returns current Unix timestamp (seconds since epoch)
<code>localtime(&now)</code>	Converts to <code>struct tm</code> in local timezone
<code>strftime</code>	Formats into "YYYY-MM-DD HH:MM:SS"

calculateRiskScore(float temp, int bp, int pulse)

int Assigns penalty points for each abnormal vital sign and returns the total composite risk score.

```
int calculateRiskScore(float temp, int bp, int pulse) {
    int score = 0;
    if (temp >= 39.5) score += 40;    // high fever
    else if (temp >= 38.0) score += 20; // mild fever
    else if (temp <= 35.0) score += 30; // hypothermia

    if (bp >= 180) score += 40;    // hypertensive crisis
    else if (bp >= 140) score += 20; // stage 2 hypertension
    else if (bp <= 90) score += 30; // hypotension / shock risk

    if (pulse >= 120) score += 30; // tachycardia
    else if (pulse <= 50) score += 30; // bradycardia
    return score;
}
```

Maximum possible score: $40 + 40 + 30 = 110$. Score $\geq 70 \rightarrow$ EMERGENCY, $\geq 35 \rightarrow$ URGENT, else NORMAL.

classifyPriority(struct Patient *p)

void Uses the already-computed `riskScore` to write the priority string into `p->priority` via `strcpy`.

```
void classifyPriority(struct Patient *p) {
    if (p->riskScore >= 70) strcpy(p->priority, "EMERGENCY");
    else if (p->riskScore >= 35) strcpy(p->priority, "URGENT");
    else
        strcpy(p->priority, "NORMAL");
}
```

Receives a **pointer** so the modification affects the original struct in the caller, not a copy.

routeDepartment(struct Patient *p)

void Converts the symptom to lowercase, then uses `strstr` keyword matching to set the appropriate department.

```
void routeDepartment(struct Patient *p) {
    char sym[100];
    strcpy(sym, p->symptom);           // safe working copy
    int i;
    for (i = 0; sym[i]; i++)           // manual tolower
        if (sym[i] >= 'A' && sym[i] <= 'Z') sym[i] += 32;

    if (strstr(sym, "chest") || strstr(sym, "heart"))
        strcpy(p->department, "Cardiology");
    else if (strstr(sym, "seizure") || strstr(sym, "headache") ...)
        strcpy(p->department, "Neurology");
    // ... etc.
}
```

Keywords

Department

chest, heart

Cardiology

seizure, dizziness, headache, neuro

Neurology

fracture, bone, joint, sprain

Orthopedics

fever, vomit, diarrhoea, abdomen

General Medicine

burn, wound, injury

Emergency Surgery

(anything else)

General OPD

logToFile(struct Patient p)

void Appends one formatted patient record to `patients.txt` using mode `"a"` so existing records are preserved.

```
void logToFile(struct Patient p) {
    FILE *fp = fopen(LOG_FILE, "a");
    if (fp == NULL) { printf("Error..."); return; }
    fprintf(fp, "[%s] ID:%d | Name:%-20s | Age:%3d | ...\n", ...);
    fclose(fp);
}
```

Note that `p` is passed **by value** (a copy) here because the function only reads, never modifies, the struct.

loadFromFile()

void Called once at startup. Opens `patients.txt`, reads every line with `fgets`, and parses each line back into a `Patient` struct using `sscanf`. This is what makes Option 2 (View All Patients) show records from previous sessions.

```
void loadFromFile() {
    FILE *fp = fopen(LOG_FILE, "r");
    if (fp == NULL) return; // first run, no file yet
    char line[400];
    while (fgets(line, sizeof(line), fp) != NULL ...) {
        struct Patient p;
        int parsed = sscanf(line, "[%[^]] ID:%d | Name:%20[^|] | ...", ...);
        if (parsed == 10) {
            // trim trailing spaces from %-20 padding
            int len = strlen(p.name) - 1;
            while (len >= 0 && p.name[len] == ' ') p.name[len--] = '\0';
            // update maxID
            if (p.id > maxID) maxID = p.id;
            patients[numPatients++] = p;
        }
    }
    fclose(fp);
}
```

`sscanf` returns the number of items successfully matched. Checking `parsed == 10` ensures only complete, well-formed lines are loaded.

rewriteLogFile()

void Opens `patients.txt` in `"w"` mode (which erases the file) and rewrites every remaining patient from `patients[]`. Called after any deletion to keep the file in sync with in-memory state.

bubbleSortByScore()

void Sorts the global `patients[]` array in **descending** order of `riskScore` (highest score = most critical = displayed first).

```
void bubbleSortByScore() {
    int i, j;
    struct Patient temp;
    for (i = 0; i < numPatients - 1; i++) {
        for (j = 0; j < numPatients - i - 1; j++) {
            if (patients[j].riskScore < patients[j+1].riskScore) {
                temp          = patients[j];          // struct assignment copies
all fields
                patients[j]  = patients[j+1];
                patients[j+1] = temp;
            }
        }
    }
}
```

Swapping two `struct Patient` variables with `temp = a; a = b; b = temp;` works because C copies every field of the struct automatically — no need to copy field by field.

addPatient()

void The full patient intake pipeline:

1. Assign a unique ID (`++maxID`)
2. Read name via `fgets + strcspn trim`
3. Read age, temperature, BP, pulse via `scanf`
4. Read symptom via `fgets + strcspn trim`
5. Call `calculateRiskScore` → `classifyPriority` → `routeDepartment` → `getCurrentTimestamp`
6. Store in `patients[numPatients++]`
7. Print summary box with colour-coded priority using `strcmp`
8. Call `logToFile` to persist

getchar() before fgets: After `scanf("%d", &choice)` in `main`, the `'\n'` is still in the buffer. The first `getchar()` in `addPatient()` consumes it so the subsequent `fgets` reads the actual name.

displayAllPatients()

void Calls `bubbleSortByScore()`, then prints a colour-coded table of all patients. Colour is selected using `strcmp` on the priority field.

```
const char *col = GREEN;
if (strcmp(patients[i].priority, "EMERGENCY") == 0) col = RED;
else if (strcmp(patients[i].priority, "URGENT") == 0) col = YELLOW;
printf("%s ... \n" RESET, col, ...);
```

The ANSI escape codes (`\033[1;31m` etc.) are defined as macros and are concatenated directly into format strings — this works because adjacent string literals are merged by the C compiler.

viewLogFile()

void Opens the file in read mode and prints raw lines as-is using `fgets` in a loop — no parsing, no sorting. Useful for auditing the exact file content.

searchPatientByName()

void Case-insensitive partial name search across all records:

1. Read query with `fgets + strcspn trim`
2. Manually lowercase the query string
3. For each patient, `strcpy` the name into a local buffer, lowercase it
4. Use `strstr(nameLower, query)` — if non-NULL, print the match

A local copy is used for lowercasing so the original `patients[i].name` (with its original casing) is never modified.

deletePatientByID()

void Deletes one patient by ID, keeping both the in-memory array and the file in sync.

1. Call `displayAllPatients()` (also sorts the array)
2. Read target ID via `scanf`
3. Linear search for that ID in the sorted array
4. Confirm with `fgets` (avoids the leftover `'\n'` bug of `scanf("%c")`)
5. Left-shift all elements after the found index by one position
6. Decrement `numPatients`
7. Call `rewriteLogFile()`

```
// Left-shift deletion:
// Before: [A][B][C][D] (delete index 1 = B)
// After shift: [A][C][D][D]
// After numPatients--: [A][C][D]
for (i = found; i < numPatients - 1; i++)
    patients[i] = patients[i + 1];
numPatients--;
```

deleteAllRecords()

void Asks for exact confirmation string "YES" using `strcmp`, then sets `numPatients = 0` and opens the file in "w" mode (which truncates it to 0 bytes) and immediately closes it.

deleteMenu()

void A sub-menu using switch-case that dispatches to `deletePatientByID()` or `deleteAllRecords()`.

displayMenu()

void Prints the 6-option main menu using `printf` with ANSI colour codes for the border.

main()

int Program entry point. Structure:

```
int main() {
    loadFromFile();           // restore past records
    do {
        displayMenu();
        scanf("%d", &choice);
        switch (choice) {
            case 1: addPatient();           break;
            case 2: displayAllPatients(); break;
            case 3: searchPatientByName(); break;
            case 4: viewLogFile();           break;
            case 5: deleteMenu();           break;
            case 6: /* exit message */      break;
            default: /* invalid */
        }
    } while (choice != 6);
    return 0;
}
```

A **do-while** loop is used (not while) so the menu is always shown at least once before checking the exit condition.

6. C Concepts Summary Table

Concept	Where Used	Key Point
struct	Patient definition, temp in sort	Groups heterogeneous data under one name
Array of structs	patients[MAX_PATIENTS]	Access fields via patients[i].field
Pointer to struct	classifyPriority, routeDepartment	Use <code>p->field</code> ; allows modification of original
char array (string)	All string fields	Must use strcpy/strcmp/strstr, not <code>=</code> / <code>==</code>
File I/O	logToFile, loadFromFile, rewriteLogFile	fopen/fprintf/fgets/fclose; modes a/r/w
Bubble sort	bubbleSortByScore	Descending sort using struct swap
if-else chain	calculateRiskScore, classifyPriority	Only the first matching branch executes
switch-case	main, deleteMenu	Cleaner dispatch than long if-else chains
do-while	main loop	Guarantees at least one execution
for loop	bubble sort, lowercase, shift-delete	Used when iteration count is known
while loop	loadFromFile, trim loops, buffer drain	Used when iteration count is unknown
Preprocessor macros	#define MAX_PATIENTS, colour codes	Replaced by the preprocessor before compilation
Global variables	patients[], numPatients, maxID	Accessible from all functions without passing as parameters
sscanf	loadFromFile	Parse structured text from a string, not from keyboard
time.h (time, localtime, strftime)	getCurrentTimestamp	Get and format current system date/time

